

[TYPE THE COMPANY NAME]

K.kalyan REG
NO:192525103

CO2(AT1)

[Type the abstract of the document here. The abstract is typically a short summary of the contents of the document. Type the abstract of the document here. The abstract is typically a short summary of the contents of the document.]

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)

Activity Tool : Analytical Problem Solving (High)

Assessment Tool Weightage: 40%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used. Bloom's Level: BL3 – Apply	BL4 – Analyze
2	Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations. Bloom's Level: BL4 – Analyze	BL5 – Evaluate
3	Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates. Bloom's Level: BL6 – Create	BL6 – Create
4	Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution. Bloom's Level: BL4 – Analyze	BL6 – Create
5	Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency. Bloom's Level: BL5 – Evaluate	BL4 – Analyze
6	Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction. Bloom's Level: BL6 – Create	BL5 – Evaluate
7	Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation. Bloom's Level: BL4 – Analyze	BL4 – Analyze

8	Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate
9	Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem. Bloom's Level: BL6 – Create	BL6 – Create
10	Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate

Code of Conduct: K.kalyan(192525103) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:k.kalyan

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	All calculations accurate;	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations

	correct final answer			
Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Q1. Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used.

SQL Query

```
SELECT S.StudentID, S.StudentName, D.DepartmentName
```

```
FROM Student S
```

```
JOIN Department D
```

```
ON S.DepartmentID = D.DepartmentID;
```

Output

StudentID	StudentName	DepartmentName
101	Arun	CSE
102	Priya	ECE
103	Rahul	CSE
104	Divya	IT

Q2. Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations.

Expression

$$\pi \text{ StudentName } (\sigma \text{ Marks} > 80 (\text{Student}))$$

Equivalent SQL

```
SELECT StudentName
```

```
FROM Student
```

```
WHERE Marks > 80;
```

Output

StudentName
Arun
Priya
Divya

Q3. Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates.

SQL Query

```
UPDATE Student
```

```
SET Marks=90
```

```
WHERE StudentID=103;
```

Output

Message

1 Row Updated

Q4. Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution.

SQL Query

```
SELECT StudentName
```

```
FROM Student
```

```
WHERE Marks>(
```

```
SELECT AVG(Marks)
```

```
FROM Student);
```

Average = 85.75

Output

StudentName
Priya
Divya

Q5. Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency.

Join Query

```
SELECT StudentName,DepartmentName
```

```
FROM Student
```

```
JOIN Department
```

```
ON Student.DepartmentID=Department.DepartmentID;
```

Subquery

```
SELECT StudentName,
```

```
(SELECT DepartmentName
```

```
FROM Department
```

```
WHERE Department.DepartmentID=Student.DepartmentID)
```

```
AS Department
```

FROM Student;

Output

StudentName	Department
Arun	CSE
Priya	ECE
Rahul	CSE
Divya	IT

Q6. Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction.

SQL Query

```
CREATE VIEW CSE_Students AS
```

```
SELECT StudentID,StudentName
```

```
FROM Student
```

```
WHERE DepartmentID=1;
```

Retrieve View

```
SELECT * FROM CSE_Students;
```

Output

StudentID	StudentName
101	Arun
103	Rahul

Q7. Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation.

Relational Algebra

$$\pi \text{ StudentName } (\sigma \text{ DepartmentID}=1(\text{Student}))$$

SQL

```
SELECT StudentName
```

```
FROM Student
```

```
WHERE DepartmentID=1;
```

Output

StudentName
Arun
Rahul

Q8. Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements

Original Query

```
SELECT *
FROM Student
WHERE DepartmentID IN
(SELECT DepartmentID
FROM Department
WHERE DepartmentName='CSE');
```

Optimized Query

```
SELECT S.*
FROM Student S
JOIN Department D
ON S.DepartmentID=D.DepartmentID
WHERE D.DepartmentName='CSE';
```

Output

StudentID	StudentName	DepartmentID	Marks
101	Arun	1	85
103	Rahul	1	78

Q9. Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem. SQL Query

```
SELECT DepartmentID,
AVG(Marks) AS AverageMarks
FROM Student
GROUP BY DepartmentID;
```

Output

DepartmentID	AverageMarks
--------------	--------------

1	81.5
2	92
3	88

Q10. Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning.

SQL Query

```
SELECT D.DepartmentName,
COUNT(S.StudentID) AS TotalStudents
FROM Department D
LEFT JOIN Student S
ON D.DepartmentID=S.DepartmentID
GROUP BY D.DepartmentName;
```

Output

DepartmentName	TotalStudents
CSE	2
ECE	1
IT	1